Puissance 4 Tridimensionnel

Rapport de projet — Programmation Orientée Objet en C++

Youssef	Responsable gestion du code
Moussa	Responsable des tests et développement des stratégies
Maher	Responsable du rapport LaTeX et co-développeur
Mohamed	Responsable documentation Doxygen et co-développeur

Année universitaire 2025–2026

Table des matières

1 Introduction **3**

2 Architecture générale du projet **3**
2.1 Vue d'ensemble des classes 3
2.2 Forme canonique — principe appliqué 3
2.3 Encapsulation — principe appliqué 4
2.4 Pattern Stratégie 4

3 Analyse classe par classe **4**
3.1 Classe `Cell` 4
3.2 Classe `Move` 4
3.3 Classe `BitBoard` 5
3.4 Classe `Board` 5
3.5 Classe abstraite `Player` 6
3.6 Classe `AIPlayer` 6
3.7 Classe `Game` 6
3.8 Classe abstraite `Evaluator` 6
3.9 Classe `AlphaBetaStrategy` 7

4 Moteur IA — techniques avancées **7**
4.1 Représentation bitboard 128 bits 7
4.2 Alpha-bêta avec approfondissement itératif 7
4.3 Table de transposition Zobrist 7
4.4 Tri des coups (*Move Ordering*) 8
4.5 Killer Moves 8
4.6 Asymétrie défensive Blancs/Noirs 8

5 Responsabilités des membres de l'équipe **8**
5.1 Youssef — Responsable gestion du code 8
5.2 Moussa — Responsable des tests et développement des stratégies . . . 9
5.3 Maher — Responsable du rapport LaTeX et co-développeur 9
5.4 Mohamed — Responsable documentation Doxygen et co-développeur . 10

6 Conformité forme canonique — tableau récapitulatif **10**

7 Conformité encapsulation — tableau récapitulatif **11**

8 Tests et validation **11**

8.1 Tests unitaires effectués 11
8.2 Partie de test IA vs IA 11

9 Conclusion **12**

1. Introduction

Ce projet consiste en l'implémentation d'un jeu de Puissance 4 tridimensionnel en C++, où un joueur humain s'affronte contre une intelligence artificielle. Le plateau de jeu comporte **4 colonnes**, **5 profondeurs** et **4 niveaux**, soit 80 cases au total. L'objectif est d'aligner 4 perles dans l'une des 122 directions possibles de l'espace 3D.

L'accent du projet est mis sur trois axes principaux.

— **Architecture orientée objet rigoureuse** : forme canonique, encapsulation, polymorphisme et pattern Stratégie.
— **Moteur IA performant** : représentation bitboard sur 128 bits, algorithme alpha-bêta avec approfondissement itératif, table de transposition Zobrist et tri des coups.
— **Qualité logicielle** : séparation nette des responsabilités, documentation Doxygen complète, et tests systématiques.

2. Architecture générale du projet

2.1. Vue d'ensemble des classes

Le projet est organisé autour de douze classes réparties en trois couches fonctionnelles distinctes.

Couche	Classes
Modèle du plateau	`Cell`, `Move`, `Board`, `BitBoard`, `Couleur`
Logique de jeu	`Player`, `AIPlayer`, `HumanPlayer`, `Game`
Moteur IA	`Strategy`, `AlphaBetaStrategy`, `Evaluator`, `EvaluatorAvance`, `EvaluatorMoyen`

2.2. Forme canonique — principe appliqué

Chaque classe non-triviale implémente la **forme canonique de Coplien** en C++, qui impose quatre éléments fondamentaux.

Forme canonique (règle des quatre) :

1. Constructeur paramétré (ou par défaut)
2. Constructeur de copie

3. Opérateur d'affectation `operator=` (avec auto-affectation `if(this!=&other)`)
4. Destructeur (virtuel pour les classes de base)

2.3. Encapsulation — principe appliqué

Tous les attributs sont déclarés `private` ou `protected`. L'accès se fait exclusivement par des méthodes publiques (*getters/setters* ou méthodes métier). Aucun attribut n'est exposé directement.

2.4. Pattern Stratégie

Le projet applique le **pattern Stratégie (GoF)** pour séparer l'algorithme de calcul du joueur IA.

— `AIPlayer` possède un pointeur `Strategy*`.
— Il appelle `strategy->meilleurCoup()` sans connaître l'algorithme sous-jacent.
— On peut substituer une stratégie sans modifier `AIPlayer`.

3. Analyse classe par classe

3.1. Classe `Cell`

`Cell` représente une case individuelle du plateau 3D.

Attributs privés	`bool m_occupee`, `Couleur m_couleur`
Méthodes publiques	`estOccupee()`, `getCouleur()`, `occuper(Couleur)`, `vider()`
Forme canonique	✓ constructeur, copie, affectation, destructeur

L'encapsulation est totale : les deux attributs sont privés et aucun accès direct n'est possible. Le constructeur initialise la case à l'état vide (`m_occupee=false`, `m_couleur=VIDE`).

3.2. Classe `Move`

`Move` encapsule un coup (colonne + profondeur). Dans la version finale, `operator==` a été ajouté pour la comparaison des *killer moves*.

```
1 Move::Move(int col, int prof) : m_colonne(col),
      m_profondeur(prof) {}
2
3 Move::Move(const Move& other)
4     : m_colonne(other.m_colonne),
          m_profondeur(other.m_profondeur) {}
5
```

```
6  Move& Move::operator=(const Move& other) {
7      if (this != &other) {
8          m_colonne    = other.m_colonne;
9          m_profondeur = other.m_profondeur;
10     }
11     return *this;
12 }
```

Listing 1 – Forme canonique de Move

3.3. Classe `BitBoard`

`BitBoard` est le *cœur technique* du projet. Elle représente le plateau via deux entiers de **128 bits** (`unsigned __int128`) : un pour les perles blanches, un pour les perles noires.

Indexation : la case $(col, prof, niv)$ correspond au bit numéro $col \times 20 + prof \times 4 + niv$.

Membres statiques : les 122 masques des lignes gagnantes et la table Zobrist sont calculés une seule fois (initialisation paresseuse) et partagés par toutes les instances.

Attributs privés	`u128 blanc`, `u128 noir`, `int niveauLibre[4][5]`, `u64 hash`
Méthodes clés	`poser()`, `annuler()`, `estGagne()`, `getHash()`
Forme canonique	✓ constructeur, copie (`memcpy`), affectation, destructeur

Do/Undo : les méthodes `poser()` et `annuler()` permettent d'explorer un coup puis de le défaire sans copier l'objet entier, ce qui représente un gain de performance majeur dans l'alpha-bêta.

Trait Zobrist : la version finale ajoute `ZOBRIST_TRAIT`, une valeur aléatoire appliquée par XOR aux nœuds MIN de l'arbre, afin de distinguer dans la table de transposition la même position selon le joueur dont c'est le tour.

3.4. Classe `Board`

`Board` maintient une **double représentation** synchronisée :

- `Cell grille[4][5][4]` pour l'affichage terminal,
- `BitBoard bitboard` pour le moteur IA.

La méthode `placerPerle()` met à jour les deux représentations simultanément, garantissant l'absence de désynchronisation. Le constructeur de copie et l'opérateur d'affectation copient les deux structures complètement.

3.5. Classe abstraite `Player`

`Player` est la classe de base abstraite du polymorphisme joueur. Elle déclare la méthode purement virtuelle `jouer(const Board&)`. Le destructeur virtuel est indispensable : `Game` détruit ses joueurs via des `Player*`, et sans lui le destructeur des sous-classes ne serait jamais appelé.

Les attributs `couleur` et `nom` sont `protected` pour être accessibles aux sous-classes sans passer par des *getters*.

3.6. Classe `AIPlayer`

`AIPlayer` hérite de `Player` et implémente le **pattern Stratégie**. Il possède un `Strategy*` dont il prend l'*ownership* et qu'il libère dans son destructeur.

```
1 AIPlayer::~AIPlayer() {
2     delete strategy;  // ownership : AIPlayer libere la
          Strategy
3 }
```

Listing 2 – Destructeur d'AIPlayer avec libération de la stratégie

Note sur le constructeur de copie : dans la version finale, le constructeur de copie effectue une copie superficielle du pointeur (`strategy = other.strategy`). Ce choix est accepté car `AIPlayer` n'est jamais copié en pratique ; la forme canonique est néanmoins complète.

3.7. Classe `Game`

`Game` orchestre la boucle de jeu. Elle prend l'*ownership* des deux `Player*` et les libère dans son destructeur.

Attributs privés	`Board plateau`, `Player* joueurs[2]`, `int tourCourant`
Méthode principale	`lancer()` : boucle affichage → coup → pose → victoire
Forme canonique	✓ constructeur, copie, affectation, destructeur

3.8. Classe abstraite `Evaluator`

`Evaluator` définit l'interface d'évaluation de position. La méthode virtuelle pure `evaluer(bb, coulIA, joueurSecond)` retourne un score relatif à `coulIA` (`>0` = bon pour l'IA). La version finale introduit le paramètre `joueurSecond` (booléen) pour activer une **asymétrie défensive** lorsque l'IA joue les Noirs.

Deux implémentations concrètes sont fournies.

— `EvaluatorAvance` : alignements partiels + fourchettes ($\approx 10\,\mu s$), utilisé aux feuilles de l'arbre.

— `EvaluatorMoyen` : alignements seuls ($\approx 0.4\,\mu s$), utilisé pour le tri des coups.

3.9. Classe `AlphaBetaStrategy`

`AlphaBetaStrategy` est l'algorithme principal de l'IA. Elle implémente la méthode `meilleurCoup()` de l'interface `Strategy` et possède ses propres `Evaluator*` dont elle prend l'*ownership*.

Les techniques avancées implémentées sont détaillées dans la section 4.

4. Moteur IA — techniques avancées

4.1. Représentation bitboard 128 bits

Chaque position est codée sur deux entiers de 128 bits. La détection de victoire se réduit à un test bitâ-bit sur 6 masques :

```
1 bool BitBoard::estGagne(int col, int prof, Couleur coul)
      const {
2     int idx = indice(col, prof, niveauLibre[col][prof] - 1);
3     u128 b  = (coul == BLANC) ? blanc : noir;
4     for (int m : MASQUES_PAR_CASE[idx])
5         if ((b & MASQUES[m]) == MASQUES[m]) return true;
6     return false;
7 }
```

Listing 3 – Détection de victoire O(1) via masques

4.2. Alpha-bêta avec approfondissement itératif

L'algorithme lance successivement des recherches alpha-bêta de profondeur $1, 2, \ldots, 10$. Le résultat du dernier niveau complet est conservé. Un délai de 7,5 secondes par coup est appliqué (avec marge de sécurité de 50 ms).

4.3. Table de transposition Zobrist

La table de transposition (`std::unordered_map`) évite de réévaluer une même position. Le hash Zobrist est mis à jour incrémentalement à chaque `poser()`/`annuler()` par une simple opération XOR.

La version finale corrige un bogue de la TT en ajoutant le **trait Zobrist** (`ZOBRIST_TRAIT`) : les nœuds MIN XORent leur hash avant consultation, ce qui distingue la même position selon le joueur actif.

4.4. Tri des coups (*Move Ordering*)

Les coups sont triés avant exploration selon quatre priorités décroissantes.

Priorité	Score
Victoire immédiate IA	200 000
Bloquer victoire adverse	150 000
Score mémorisé (TT)	score TT
EvaluatorMoyen	score rapide

4.5. Killer Moves

Deux *killer moves* par niveau de profondeur (tableau `killers[20][2]`) sont mémorisés. Un coup qui provoque une coupure bêta sans être une victoire directe est enregistré comme *killer* et prioritaire dans l'exploration ultérieure.

4.6. Asymétrie défensive Blancs/Noirs

Les Noirs (second joueur) activent des poids défensifs renforcés dans les évaluateurs. Cette asymétrie compense le désavantage stratégique d'être le deuxième à jouer.

5. Responsabilités des membres de l'équipe

5.1. Youssef — Responsable gestion du code

Périmètre : organisation du dépôt, gestion des versions et intégrité structurelle de l'ensemble du code source.

Points clés du travail :

- Mise en œuvre de la séparation `.h`/`.cpp` pour toutes les classes, en accord avec les conventions du projet.
- Vérification systématique de la forme canonique sur chaque classe : constructeur de copie, `operator=` avec auto-affectation, et destructeur virtuel pour les classes de base polymorphes.
- Contrôle de l'encapsulation : audit de chaque attribut pour s'assurer qu'aucun n'est `public`, et que les accesseurs exposent exactement ce qui est nécessaire (préférence des références constantes pour éviter les copies).
- Écriture du `Makefile` avec gestion des dépendances et règle `clean` pour la reproductibilité des compilations.
- Coordination entre les branches de développement pour éviter les conflits (notamment entre l'évolution du `BitBoard` et l'`AlphaBetaStrategy`).

Décision technique notable : Youssef a structuré la séparation stricte entre couche

affichage (`Board`) et couche calcul (`BitBoard`), garantissant qu'`AlphaBetaStrategy` ne reçoit jamais un `Board` directement.

5.2. Moussa — Responsable des tests et développement des stratégies

Périmètre : implémentation de l'algorithme alpha-bêta et validation par les tests.

Points clés du travail :

- Implémentation de l'alpha-bêta de base avec coupures α et β.
- Ajout de l'approfondissement itératif (*Iterative Deepening*) : recherche à profondeur $1, 2, \ldots, 10$ avec conservation du dernier résultat complet en cas de dépassement du temps.
- Intégration de la table de transposition : types `EXACT`/`LOWER_BOUND`/`UPPER_BOUND` et correction du bogue de trait Zobrist (`ZOBRIST_TRAIT`).
- Implémentation des *killer moves* : tableau `killers[20][2]`, enregistrement lors des coupures et priorité dans `trierCoups()`.
- Tests de non-régression : parties IA vs IA, vérification que l'IA détecte et joue toujours les victoires immédiates au premier coup.
- Tests de performance : mesure du nombre de nœuds explorés avant et après le tri des coups pour valider le gain.

Décision technique notable : le passage de la copie de `BitBoard` à la stratégie Do/Undo (`poser()`/`annuler()`) réduit la mémoire allouée dans la récursion d'un facteur 20 environ.

5.3. Maher — Responsable du rapport LaTeX et co-développeur

Périmètre : rédaction du rapport et contribution au développement des évaluateurs.

Points clés du travail :

- Structure et rédaction du présent rapport en LaTeX : plan, mise en forme des sections, tableaux récapitulatifs et extraits de code.
- Implémentation de la fonction `scoreAlignementsMasques` : parcours des 122 masques, comptage des bits alignés par couleur et score relatif à `coulIA`.
- Implémentation de `scoreFourchettes` : détection des cases accessibles avec au moins deux menaces actives, penalisation symétrique des menaces adverses.
- Ajout du paramètre `joueurSecond` dans les évaluateurs pour l'asymétrie défensive Blancs/Noirs.
- Refactoring de la hiérarchie `Evaluator` : extraction de la classe abstraite, séparation `EvaluatorAvance`/`EvaluatorMoyen`.

Décision technique notable : le score est *toujours relatif* à `coulIA`, éliminant tout gymnastic de signes dans l'alpha-bêta et rendant le code plus lisible.

5.4. Mohamed — Responsable documentation Doxygen et co-développeur

Périmètre : documentation Doxygen intégrale et contribution au développement des classes de base.

Points clés du travail :

- Documentation Doxygen complète de l'ensemble des fichiers : `@file`, `@brief`, `@param`, `@return` sur chaque méthode publique.
- Annotation des choix de conception (PATTERN STRATÉGIE, DOUBLE REPRÉSENTATION, FORME CANONIQUE) directement dans les commentaires pour faciliter la relecture.
- Implémentation de la forme canonique des classes `Cell`, `Move`, `Board` et `Player` : constructeurs de copie avec copie membre par membre, `operator=` avec garde `if(this!=&other)`.
- Vérification que `operator=` retourne bien `*this` (référence à l'objet courant) pour permettre les chaînages.
- Mise en place des membres statiques `inline` dans `BitBoard` (version précédente) puis migration vers les définitions en `.cpp` dans la version finale.

Décision technique notable : les méthodes statiques `initMasques()` et `initZobrist()` sont `private`, conformément au principe d'encapsulation — leur appel est interné dans le constructeur par une initialisation paresseuse.

6. Conformité forme canonique — tableau récapitulatif

Classe	Constructeur	Copie	Affectation	Destructeur
`Cell`	✓	✓	✓	✓
`Move`	✓	✓	✓	✓
`BitBoard`	✓	✓	✓	✓
`Board`	✓	✓	✓	✓
`Player`	✓	✓	✓	✓ (virtuel)
`AIPlayer`	✓	✓	✓	✓ (virtuel)
`Game`	✓	✓	✓	✓
`Evaluator`	✓	✓	✓	✓ (virtuel)
`EvaluatorAvance`	✓	✓	✓	✓
`EvaluatorMoyen`	✓	✓	✓	✓
`AlphaBetaStrategy`	✓	✓	✓	✓

Toutes les classes respectent intégralement la forme canonique de Coplien. Chaque

`operator=` implémente la garde contre l'auto-affectation (`if(this!=&other)`) et retourne `*this`.

7. Conformité encapsulation — tableau récapitulatif

Classe	Attributs	Accès
`Cell`	`m_occupee`, `m_couleur`	`private`
`Move`	`m_colonne`, `m_profondeur`	`private`
`BitBoard`	`blanc`, `noir`, `niveauLibre`, `hash` + statiques	`private`
`Board`	`grille[4][5][4]`, `niveauLibre`, `bitboard`	`private`
`Player`	`couleur`, `nom`	`protected`
`AIPlayer`	`strategy`	`private`
`Game`	`plateau`, `joueurs[2]`, `tourCourant`	`private`
`AlphaBetaStrategy`	tous les attributs de calcul	`private`

Note : les attributs de `Player` (`couleur`, `nom`) sont `protected` et non `private`, ce qui est justifié par l'héritage — `AIPlayer` y accède directement dans `jouer()`.

8. Tests et validation

8.1. Tests unitaires effectués

- **BitBoard** : vérification de la détection des 122 masques (comptage en fin d'initialisation), tests `estGagne()` sur les 13 directions de l'espace.
- **Do/Undo** : test que `poser()` suivi de `annuler()` restaure exactement le hash Zobrist initial (propriété XOR).
- **AlphaBetaStrategy** : test que l'IA joue systématiquement une victoire immédiate lorsqu'elle est disponible (profondeur 1).
- **Form canonique** : instanciation, copie et affectation de chaque classe pour valider l'absence de fuite mémoire.

8.2. Partie de test IA vs IA

Une partie où les deux joueurs sont des `AIPlayer` utilisant `AlphaBetaStrategy` a été exécutée pour valider :

- l'absence de coup invalide (bâtonnet plein),
- la détection correcte de la victoire en fin de partie,
- la stabilité du hash Zobrist après une partie complète.

9. Conclusion

Le projet Puissance 4 tridimensionnel a été mené à terme avec une architecture C++ rigoureuse. Les points forts de la version finale sont les suivants.

- **Forme canonique complète** sur l'ensemble des onze classes.
- **Encapsulation stricte** : aucun attribut public, accès contrôlé par des méthodes.
- **Polymorphisme correct** : destructeurs virtuels sur toutes les classes de base, `delete` via pointeur de base sûr.
- **Moteur IA performant** : bitboard 128 bits, alpha-bêta avec approfondissement itératif, TT Zobrist avec trait, killer moves et asymétrie défensive.
- **Séparation des responsabilités** : affichage (`Board`), calcul (`BitBoard`), stratégie (`AlphaBetaStrategy`), évaluation (`Evaluator`).

La seule limite identifiée concerne le constructeur de copie de `AIPlayer`, qui effectue une copie superficielle du pointeur `Strategy*`. Ce point n'est pas problématique en pratique mais pourrait être amélioré par un mécanisme de clonage (`virtual Strategy* clone() const`).